GitHub 운영 가이드

group2-project (kimchi_labeler) · .gitignore, 코드/데이터 분리, 팀원 매일 작업 흐름

1. 한눈에 보기

구분	어디에 두나	GitHub에 올리나
코드 (kimchi_labeler 프로그램)	저장소 하나만 두고 공유	올림 (feature 브랜치 → PR → main)
사진 · 라벨 · 작업 데이터	각자 PC(WSL)의 `data/` 와 `settings.json`	**절대 올리지 않음** (저작권 · NDA)
제출용 manifests/dataset_manifest.csv	저장소	올려도 됨
.gitignore	저장소 파일 (main에 한 번 들어가면 모두에게 적용)	올림

2. .gitignore 는 브랜치에 "적용"하는 게 아니다

.gitignore 는 저장소에 들어 있는 평범한 파일입니다. main 에 한 번 들어가면 모든 팀원이 받아서 같이 적용됩니다. 해야 할 일은 세 가지입니다.

- PM 문서에 적힌 .gitignore 가 main 에 들어가 있어야 합니다.
- **__pycache__/** 와 ***.pyc** 를 추가합니다. (실행하면 자동 생성되는 캐시라 올릴 필요가 없습니다.)
- main 에 PR 로 merge 하고, 팀원은 **git pull origin main** 으로 받습니다. 이미 쓰던 브랜치는 **git merge main** 으로 가져갑니다.

권장 .gitignore 내용

```
**/data/
**/이물검출_학습데이터*/
*.jpg
*.jpeg
*.zip
.venv/
__pycache__/
*.pyc
settings.json
```

> **주의: .gitignore 는 아직 추적되지 않은 파일만 막습니다**
> 이미 한 번 커밋된 파일은 나중에 .gitignore 에 써도 계속 올라갑니다. 저장소가 Public 이라 이미 올라갔다면 공개 상태입니다. 아래 명령으로 확인하세요.

```
git ls-files | grep -E "\.(jpg|jpeg|zip|pyc)$|settings.json"
```

아무것도 안 나오면 안전합니다. 무언가 나오면 **git rm --cached 파일명** 으로 추적을 해제한 뒤 커밋합니다.

3. 각자 파일을 따로 갖는 방식은 이렇게

- **코드**: 팀원마다 복사본을 만들어 올리면 버전이 갈라지고 충돌이 납니다. 한 저장소에서 담당자가 자기 feature/ 브랜치에서 수정하고 PR 로 합칩니다.
- **작업 결과(사진 · 라벨 · 작업 데이터)**: 각자 자기 PC 의 WSL 에서 data/ 와 settings.json 만 따로 갖고 작업합니다. .gitignore 가 막아 주므로 실수로 올라가지 않습니다.
- **올려도 되는 것**: 제출용 manifests/dataset_manifest.csv 정도입니다.

4. 팀원이 매일 하는 흐름

아래 순서를 그대로 따라 하면 됩니다. 각자 자기 브랜치에서만 작업하고, main 에는 직접 push 하지 않습니다.

```
git checkout 내브랜치
git pull origin main        # 최신 코드와 .gitignore 받기
# ... 각자 로컬에서 라벨링 작업 (data/ 는 올라가지 않음)
git add .
git status                  # 올라갈 파일에 사진/설정이 없는지 확인
git commit -m "작업 내용"
git push origin 내브랜치
```

- push 한 뒤 GitHub 에서 **Pull request** 를 만듭니다.
- PM 이 Files changed 를 확인하고 merge 합니다.
- merge 후 팀원은 **git checkout main** → **git pull origin main** 으로 최신 코드를 받습니다.

> **commit 하기 전 꼭 확인**
> **git status** 에 .jpg / .zip / .pyc / settings.json / .venv 가 보이면 **git add 하지 말고** 멈추세요. .gitignore 가 최신인지 확인합니다.

5. PM 이 할 일

할 일	방법
main 보호 확인	Settings → Rules → Rulesets 에서 protect-main 이 Active 인지 확인 (Classic 규칙 화면은 비어 있는 게 정상)
PR 확인 후 merge	Files changed 에서 사진 · .venv · settings.json · .pyc 가 섞이지 않았는지 보고 Merge pull request
브랜치 자동 정리	Settings → General → Pull Requests → Automatically delete head branches 켜기
쓸모없는 브랜치 삭제	Ahead 0 이거나 이미 merge 된 브랜치만 삭제. Ahead 1 이상은 먼저 PR/확인 후 삭제
팀원 초대 확인	Settings → Collaborators 에서 6명 모두 수락(Pending 아님) 상태인지 확인

현재 브랜치 정리 기준 (화면 기준)

브랜치	상태	판단
main	기본 브랜치	삭제 금지
feature/bbox, feature/qa	Ahead 0	정리해도 안전한 후보
feature/pm, feature/manifest	Ahead 1	merge 안 된 커밋 있음 → 먼저 PR
feature/gui	Behind 10, Ahead 2	오래됨 · merge 안 된 커밋 있음 → 확인 후 정리
feature/kimchi-labeler	프로그램 PR 브랜치	PR merge 후 삭제

삭제 전에 단톡방에 "이 브랜치 쓰는 사람?" 하고 공지하면 안전합니다. 삭제 직후라면 화면의 Restore 버튼으로 복구할 수 있습니다.

> **한계**
> 개인 계정 저장소에서는 "이 사람은 이 브랜치만 push 가능" 같은 사용자별 제한을 걸 수 없습니다(Organization 저장소에서만 가능). "자기 브랜치만 쓰기"는 약속으로 운영하고, main 은 규칙으로 보호합니다.